Non static Block OR Instance Initializer :

* It is a special block in java which we write inside a class and this non sttaic block will be automatically executed when we create an object for the given class.

Example :

```
public class Test
{
   {
      //Non static block   [Whenever we will create the object for Test then non static block will be executed]
   }
}
```

//Program

```
package com.ravi.nsb;

class Test
{
    {
        IO.println("Non static block");
    }
}

public class NSBDemo1
{
    public static void main(String[] args)
    {
        new Test();
        new Test();
        new Test();
    }

}
```

Note : Every time we will create the object non static block wil be executed

* The main purpose of non static block to initialize the non static field (vizag) that is the reason it is also known as **instance initializer** as well as we can provide a common message for all the objects.

//Program

```
package com.ravi.nsb;

class Demo
{
    int x;

    {
        x = 100;
        IO.println("Object creation is in process"); //Common message for all objects
    }

}


public class NSBDemo2
{
    public static void main(String[] args)
    {
        Demo d1 = new Demo();
        IO.println(d1.x);

        Demo d2 = new Demo();
        IO.println(d2.x);

        Demo d3 = new Demo();
        IO.println(d3.x);

    }

}
```

* Java compiler will automatically place all the non static blocks to the 2nd line of constructor body, If and only if, first line contains super() [If constructor first line is super() then compulsory 2nd line will be reserved for non static block]

Example :

Demo.java

```
public class Demo
{
   public Demo()
   {
      IO.println("No args constr");
   }

   {
      IO.println("NSB");
   }
}
```

javac

Demo.class

```
public class Demo
{
   public Demo()
   {
     super();  //1st line added by javac

     {
      IO.println("NSB");   //2nd line
     }

     IO.println("No args constr");
   }
}
```

```
package com.ravi.nsb;

class Sample
{
    public Sample()
    {
        super();

        IO.println("No Argument Constructor");
    }

    {
        IO.println("Non static block");
    }
}

public class NSBDemo3
{
    public static void main(String[] args)
    {
        new Sample();
        IO.println("...........");
        new Sample();
    }
}
```

In this program in the first line of constructor javac will add super() then automatically 2nd line is reserved for non static block.

* The non static block **will not be added** to the constructor which contains this() in the first line as shown in the program below.

```
package com.ravi.nsb;

class Order
{
    public Order(String foodName)
    {
        this(foodName, 2);
        //Non static block will not be added
    }

    public Order(String foodName, int quantity)
    {
        this(foodName, 2, 450);
        //Non static block will not be added
    }

    public Order(String foodName, int quantity, double price)
    {
        super();
        IO.println("Food Name is :"+foodName);
        IO.println("Food Quantity is :"+quantity+" plates");
        IO.println("Food price is RS :"+price+" per plate");
    }

    {
        IO.println("NSB");
    }
}
public class NSBDemo3
{
    public static void main(String[] args)
    {
        new Order("Paneer Butter");


    }
}
```

* If a class conatins multiple non static blocks then all these non static blocks will be executed from top to bottom [according to the order]

```
package com.ravi.nsb;

class Foo
{
    int x;

    public Foo()
    {
        this.x = 100;
        IO.println("x value is :"+x);
    }

    {
        this.x = 15;
        IO.println("x value is :"+x);
    }

    {
        this.x = 9;
        IO.println("x value is :"+x);
    }

    {
        this.x = 75;
        IO.println("x value is :"+x);
    }
}


public class NSBDemo4
{
    public static void main(String[] args)
    {
        new Foo();

    }

}
```

* All the non static blocks are executed inside the constructor body from top to bottom

* All the non static blocks are executed before the constructor body.

1) super()
2) NSB
3) Constructor Body

1) this()
2) Constructor Body

* We cannot write return statement inside non static block because all the non static blocks are executed normally without any interruption.

```
package com.ravi.nsb;

public class NSBDemo5
{
    public static void main(String[] args)
    {
    }


    {
        IO.println("Non static block");
        return;
    }

}
```

```
package com.ravi.nsb;

class Student
{

    {
        IO.println("Non static block");
        new Student();  //java.lang.StackOverflowerror
    }
}
public class NSBDemo5
{
    public static void main(String[] args)
    {
        new Student();
    }
}
```

Initialization Order of non static fields :

Case 1 :

```
class Test
{
  //Non static fields

  public Test() //Constr
  {

  }

  //Non static block
}
```

javac

```
class Test
{
  public Test() //Constructor
  {
   super();  //1st line

   //Non static fields   //Non static block //2nd line

   constructor body execution starts
  }

}
```

Case 2 :

```
class Test
{
  //Non static block

  public Test() //Constr
  {

  }

  //Non static fields
}
```

javac

```
class Test
{
  public Test() //Constructor
  {
   super();  //1st line

  //Non static block     //Non static fields

   constructor body execution starts
  }

}
```

Non static field initialization order :

Default value [by new keyword] => At the time of declaration and non static block (Priority Basis, both are having same priority) => in the body of constructor (Object creation done)

```
class Test
{
    {
        x = 200;
    }

    int x = 100;
}


public class InitializationOrder
{
    public static void main(String[] args)
    {
        Test t1 = new Test();
        IO.println(t1.x);
    }
}
```